

libutils: bugs and UB audit

Date: October 4, 2026 · **Sources:** `src/` , `include/utils/` , `tests/` , sanitizer logs (`sanitize_build.sh` `all`), maximum warnings build, 30x repeat of the threaded suites

How to read this document

1. Summary.

2. Method: tools, iterations, what was not run.

3. Sanitizer reports and why they are false positives.

4. Dependencies (security).

Report only. Scope: the whole project (commit `2b46e19` + the working tree changes), single agent, loop until no new finding (max 5). The vendored `algorithms/` are covered by the sanitizers and tests only.

Contents

1. Summary

2. Method

3. Sanitizer reports

4. Dependencies

5. Sources

1. Summary

Category	High	Medium	Low
UB / memory	0	0	0
Concurrency	0	0	0
Errors / process	0	0	0
Security / sizes	0	0	0
Total	0	0	0

Point	Result
ASan + UBSan	0 errors, 854 / 854 tests pass
LSan	24 reports, all from the intentional <code>DetectsLeak</code> tests (false positives)
TSan	2 reports in <code>SharedMemoryTest.ReadableLastOnly</code> , false positives of the test setup (see 3)
Warnings (max set)	0 in libutils' own code; 14 in the vendored <code>c2dmp-hsm</code>
Stability	threaded / process suites (<code>ClientServer</code> , <code>SharedMemory</code> , <code>Process</code> , <code>Verbose</code> , <code>Cli</code> hooks, <code>Scheduler</code> , <code>LoadBalancer</code>): 30 runs x 111 tests, 0 failure
Dependencies	0 advisory

Verdict: no bug found. Every sanitizer report is an explained false positive.

2. Method

Item	Value
Iterations	3, stopped after the 3rd found nothing new: 1 sanitizers + code review (process, network, IPC, crypto, parsers); 2 maximum warnings build + review of the parsing / ctype code and the tests; 3 30x stress of the threaded suites

Item	Value
Sanitizers	ASan + UBSan Debug build, then TSan build, clang 22.1.8, ctest with halt_on_error=0
Warnings	-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion -Wnull-dereference -Wdouble-promotion -Wformat=2 -Wcast-align -Wold-style-cast -Wthread-safety -Wunreachable-code -Wloop-analysis
Review checklist	UB / memory, leaks / resources, concurrency, errors, logic, security (audit-bugs checklist) on every src/ module
Not run	clang-tidy , cppcheck , scan-build (not installed: dnf install clang-tools-extra cppcheck clang-analyzer), valgrind (ASan instead), Optimized / Asan release builds

3. Sanitizer reports

Report	Verdict	Reason
TSan: 2 data races in SharedMemoryTest.ReadableLastOnly (send_ SharedMemory.cpp:312,322 vs read_ :126,132)	false positive	the test maps the same shared memory twice in one process (server and client instances, 2 mmap). TSan tracks the atomics by virtual address: the slot release done by the client's reader thread on its mapping (reader / flag atomics) is invisible from the server's mapping, so the chain reader fetch_sub(release) → releaser acq_rel → writer acquire isn't seen. Real users map it once per process. A clean TSan run is possible by running the client in a fork ed child
LSan: 24 leaks	false positive	intentional leaks of tests/security/observer/{Observer,UnsafeObserver,MemoryLeakNotifier}.cpp DetectsLeak (suppression leak:*DetectsLeak*)

4. Dependencies

Dependency	Version	Advisories	Used APIs
openssl-libs / openssl-devel	3.5.9-1.fc44	none	EVP_aes_256_gcm , EVP_PKEY_decrypt (RSA OAEP), EVP_EncodeBlock / EVP_DecodeBlock , RAND_bytes

Details in audit/2026-10-04-dependencies.md .

5. Sources

Skills: audit-bugs (checklist, sanitizer script), libutils-exception , cpp-tests .

Tools: audit-bugs/scripts/sanitize_build.sh , clang 22.1.8 ASan / UBSan / LSan / TSan, ctest , --gtest_repeat=30 , audit-deps/scripts/vulns.py .

Related: audit/2026-10-04-tests.md , audit/2026-10-04-quality.md , audit/2026-10-04-dependencies.md .